In [1]:
```python
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import ElasticNet, LogisticRegression
from sklearn.metrics import make_scorer, root_mean_squared_error, mean_absolute_error, precision_score, recall_score, r2_score, confusion_matrix, f1_score, roc_curve, roc_auc_score
from sklearn.model_selection import cross_val_score

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

import optuna
```

Регрессия

In [2]:
```python
RAW_FEATS = 'regression_raw_feats.csv'
RAW_TARGET = 'regression_raw_target.csv'
PROCESSED_FEATS = 'regression_processed_feats.csv'
PROCESSED_TARGET = 'regression_processed_target.csv'
```

In [3]:
```python
df_feats_raw = pd.read_csv(RAW_FEATS)
df_target_raw = pd.read_csv(RAW_TARGET)
df_feats_processed = pd.read_csv(PROCESSED_FEATS)
df_target_processed = pd.read_csv(PROCESSED_TARGET)
```

In [4]:
```python
def evaluate(metrics, y_test, y_pred, round_num):
    for metric in metrics:
        print(f'Model score for metric "{metric.__name__}": {round(metric(y_test, y_pred), round_num)}')

    plt.scatter(y_test, y_pred, alpha=0.7, label='Predictions')
    line_coords = np.linspace(np.min(y_test), np.max(y_test), 100)
    plt.plot(line_coords, line_coords, color='red', linestyle='--', linewidth=2, label='Perfect prediction')
    plt.xlabel('y_test')
    plt.ylabel('y_pred')
    plt.title('Actual vs predicted values')
    plt.legend()
    plt.grid(True)
    plt.show()
```

In [5]:
```python
eval_metrics = [mean_absolute_error, root_mean_squared_error, r2_score]
random_state = 42
test_size = 0.2

round_num = 4

optuna_metric = root_mean_squared_error
optuna_direction = "minimize"
n_trials = 30
cv = 5
```

Готовая модель

In [6]:
```python
model_class = ElasticNet
```

Бейзлайн

In [ ]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_raw, df_target_raw, test_size = test_size, random_state = random_state)

y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

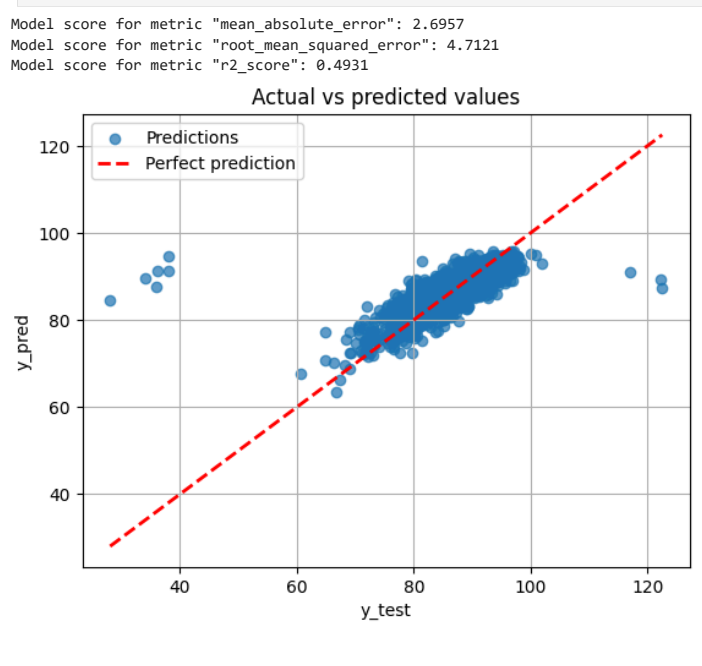
In [8]:
```python
model = model_class()

model.fit(X_train, y_train)
```

Out[8]:
```
ElasticNet
ElasticNet()
```

In [9]:
```python
y_pred = model.predict(X_test)
```

In [10]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.6957
Model score for metric "root_mean_squared_error": 4.7121
Model score for metric "r2_score": 0.4931
```



Улучшенный бейзлайн

Улучшение за счёт препроцессинга

In [ ]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_processed, df_target_processed, test_size = test_size, random_state = random_state)

y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

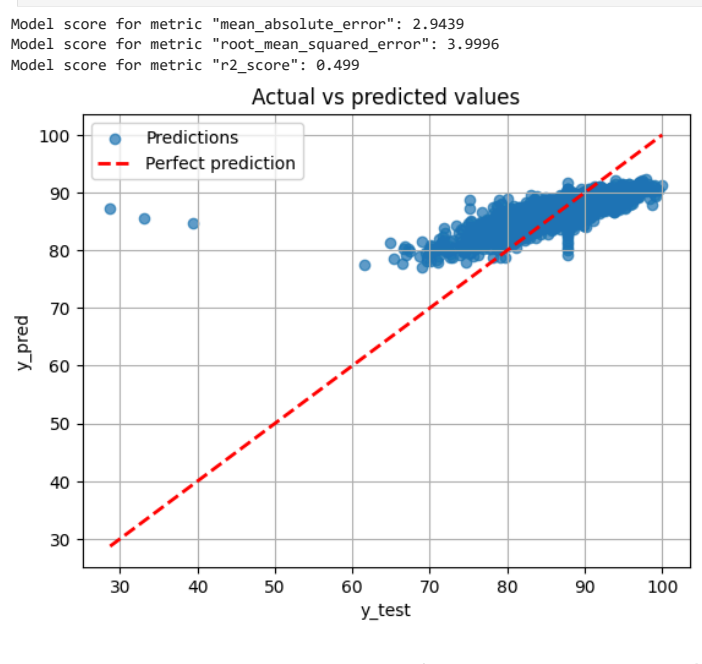
In [12]:
```python
model = model_class()

model.fit(X_train, y_train)
```

Out[12]:
```
ElasticNet
ElasticNet()
```

In [13]:
```python
y_pred = model.predict(X_test)
```

In [14]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.9439
Model score for metric "root_mean_squared_error": 3.9996
Model score for metric "r2_score": 0.499
```



Дополнительное улучшение за счёт подбора параметров на cross-val

In [15]:
```python
def objective(trial):
    optuna_params = {
        'alpha': trial.suggest_float('alpha', 0.0, 1.0),
        'l1_ratio': trial.suggest_float('l1_ratio', 0.0, 1.0),
        'max_iter': trial.suggest_int('max_iter', 1000, 5000),
        'tol': trial.suggest_float('tol', 1e-5, 1e-3),
    }

    trial_model = model_class(**optuna_params)

    score = cross_val_score(trial_model, X_train, y_train, cv=cv, scoring=make_scorer(optuna_metric), n_jobs=-1).mean()

    return score

study = optuna.create_study(direction=optuna_direction, sampler=optuna.samplers.TPESampler(seed=random_state))
study.optimize(objective, n_trials=n_trials)

print(f"Лучшие параметры для модели: {study.best_params}")
print(f"Лучшая метрика ({optuna_metric.__name__}): {round(study.best_value,round_num)}")

best_model = model_class(**study.best_params)
best_model.fit(X_train, y_train)
```

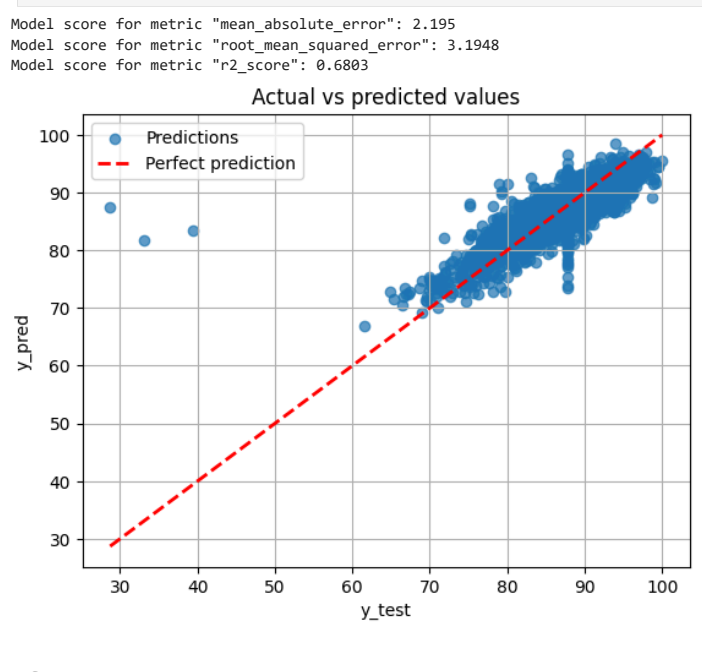
```
[I 2025-12-08 22:52:39,931] A new study created in memory with name: no-name-4e6e88d7-9c8d-4b95-ae08-57c6dd23197e
[I 2025-12-08 22:52:41,379] Trial 0 finished with value: 4.485771924274385 and parameters: {'alpha': 0.3745401188473625, 'l1_ratio': 0.9507143064099162, 'max_iter': 3928, 'tol': 0.0006026718993550663}. Best is trial 0 with value: 4.485771924274385.
[I 2025-12-08 22:52:42,374] Trial 1 finished with value: 4.394423356210999 and parameters: {'alpha': 0.15601864044243652, 'l1_ratio': 0.15599452033620265, 'max_iter': 1232, 'tol': 0.0008675143843171858}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,284] Trial 2 finished with value: 4.62589675888964 and parameters: {'alpha': 0.6011150117432088, 'l1_ratio': 0.7080725777960455, 'max_iter': 1082, 'tol': 0.0009702107536403744}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,314] Trial 3 finished with value: 4.833513006758638 and parameters: {'alpha': 0.8324426408004217, 'l1_ratio': 0.21233911067827616, 'max_iter': 1727, 'tol': 0.00019157046475489948}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,346] Trial 4 finished with value: 4.463772176736875 and parameters: {'alpha': 0.3042422429595377, 'l1_ratio': 0.5247564316322378, 'max_iter': 2728, 'tol': 0.0002983168487960615}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,378] Trial 5 finished with value: 4.699753866186997 and parameters: {'alpha': 0.6118528947223795, 'l1_ratio': 0.13949386065204183, 'max_iter': 2168, 'tol': 0.00037269822486075483}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,409] Trial 6 finished with value: 4.53771915033064 and parameters: {'alpha': 0.45606998421703593, 'l1_ratio': 0.7851759613930136, 'max_iter': 1798, 'tol': 0.0005190920940294755}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,441] Trial 7 finished with value: 4.696863436527883 and parameters: {'alpha': 0.5924145688620425, 'l1_ratio': 0.046450412719997725, 'max_iter': 3430, 'tol': 0.0001788188824504186}. Best is trial 1 with value: 4.394423356210999.
[I 2025-12-08 22:52:43,473] Trial 8 finished with value: 4.3368057647879725 and parameters: {'alpha': 0.06505159298527952, 'l1_ratio': 0.9488855372533332, 'max_iter': 4863, 'tol': 0.0008103133746352965}. Best is trial 8 with value: 4.3368057647879725.
[I 2025-12-08 22:52:43,504] Trial 9 finished with value: 4.493447301125125 and parameters: {'alpha': 0.3046137691733707, 'l1_ratio': 0.09767211400638387, 'max_iter': 3737, 'tol': 0.0004457509688022053}. Best is trial 8 with value: 4.3368057647879725.
[I 2025-12-08 22:52:43,535] Trial 10 finished with value: 4.331244012352988 and parameters: {'alpha': 0.020546809824298393, 'l1_ratio': 0.47332732145466855, 'max_iter': 4879, 'tol': 0.0007125434479173731}. Best is trial 10 with value: 4.33124401235298
8.
[I 2025-12-08 22:52:43,566] Trial 11 finished with value: 4.3305020028109675 and parameters: {'alpha': 0.01454087516236438, 'l1_ratio': 0.4016647941567012, 'max_iter': 4949, 'tol': 0.0007634576416083034}. Best is trial 11 with value: 4.330502002810967
5.
[I 2025-12-08 22:52:43,598] Trial 12 finished with value: 4.334429616243351 and parameters: {'alpha': 0.03435746358874557, 'l1_ratio': 0.3796042434202235, 'max_iter': 4984, 'tol': 0.0006826975469036963}. Best is trial 11 with value: 4.330502002810967
5.
[I 2025-12-08 22:52:43,630] Trial 13 finished with value: 4.394929455210344 and parameters: {'alpha': 0.17550991593332854, 'l1_ratio': 0.41508030158786696, 'max_iter': 4432, 'tol': 0.0007283047566480887}. Best is trial 11 with value: 4.330502002810967
5.
[I 2025-12-08 22:52:43,661] Trial 14 finished with value: 4.329940630251746 and parameters: {'alpha': 0.010813209728300745, 'l1_ratio': 0.5760103475555451, 'max_iter': 4330, 'tol': 0.000937733682004973}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,693] Trial 15 finished with value: 4.824334687064775 and parameters: {'alpha': 0.9119167232179372, 'l1_ratio': 0.6895366011206421, 'max_iter': 4327, 'tol': 0.0009500202567910115}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,725] Trial 16 finished with value: 4.404001066660217 and parameters: {'alpha': 0.1830610799292295, 'l1_ratio': 0.29974150227632423, 'max_iter': 3068, 'tol': 0.0008267541706232683}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,756] Trial 17 finished with value: 4.426447181400886 and parameters: {'alpha': 0.24387496086356825, 'l1_ratio': 0.5586366447622302, 'max_iter': 4276, 'tol': 0.0009830672492423369}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,802] Trial 18 finished with value: 4.350044170043773 and parameters: {'alpha': 0.09039590229092494, 'l1_ratio': 0.5864590448357907, 'max_iter': 3946, 'tol': 0.0005542483582499034}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,833] Trial 19 finished with value: 4.760656809554932 and parameters: {'alpha': 0.7298348295317014, 'l1_ratio': 0.30344630088855795, 'max_iter': 4638, 'tol': 0.0008808177874592472}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,864] Trial 20 finished with value: 4.514647229593029 and parameters: {'alpha': 0.4134140996927128, 'l1_ratio': 0.8152590804011588, 'max_iter': 3235, 'tol': 0.0006332449750962679}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,896] Trial 21 finished with value: 4.334759308418086 and parameters: {'alpha': 0.03720314284198487, 'l1_ratio': 0.4456913325009604, 'max_iter': 4995, 'tol': 0.0007508122756340961}. Best is trial 14 with value: 4.329940630251746.
[I 2025-12-08 22:52:43,928] Trial 22 finished with value: 4.329912631077085 and parameters: {'alpha': 0.010989189243771236, 'l1_ratio': 0.6393870521051082, 'max_iter': 4568, 'tol': 0.0007492947161424528}. Best is trial 22 with value: 4.3299126310770
85.
[I 2025-12-08 22:52:43,960] Trial 23 finished with value: 4.370044444386042 and parameters: {'alpha': 0.14158424165235128, 'l1_ratio': 0.6423620561394953, 'max_iter': 4086, 'tol': 3.070443081856989e-05}. Best is trial 22 with value: 4.32991263107708
5.
[I 2025-12-08 22:52:43,992] Trial 24 finished with value: 4.3294290805244335 and parameters: {'alpha': 0.0003546890980077183, 'l1_ratio': 0.79903868834662, 'max_iter': 4612, 'tol': 0.000784906091545598}. Best is trial 24 with value: 4.329429080524433
5.
[I 2025-12-08 22:52:44,023] Trial 25 finished with value: 4.43047682720109 and parameters: {'alpha': 0.2661907052857681, 'l1_ratio': 0.8420867358525691, 'max_iter': 3666, 'tol': 0.0008923922551398093}. Best is trial 24 with value: 4.3294290805244335.
[I 2025-12-08 22:52:44,054] Trial 26 finished with value: 4.358014201522903 and parameters: {'alpha': 0.11994792988037481, 'l1_ratio': 0.7394147599861284, 'max_iter': 4518, 'tol': 0.0006437772418893812}. Best is trial 24 with value: 4.329429080524433
5.
[I 2025-12-08 22:52:44,086] Trial 27 finished with value: 4.414398142004497 and parameters: {'alpha': 0.22768745673289145, 'l1_ratio': 0.6274198266149676, 'max_iter': 4624, 'tol': 0.0007995717608725975}. Best is trial 24 with value: 4.329429080524433
5.
[I 2025-12-08 22:52:44,118] Trial 28 finished with value: 4.349100595938765 and parameters: {'alpha': 0.09222316232528953, 'l1_ratio': 0.6677924495840926, 'max_iter': 2508, 'tol': 0.0009050689171865545}. Best is trial 24 with value: 4.329429080524433
5.
[I 2025-12-08 22:52:44,150] Trial 29 finished with value: 4.487379618585654 and parameters: {'alpha': 0.3682458851596138, 'l1_ratio': 0.9057271321920898, 'max_iter': 4010, 'tol': 0.000590567392374896}. Best is trial 24 with value: 4.3294290805244335.
Лучшие параметры для модели: {'alpha': 0.0003546890980077183, 'l1_ratio': 0.79903868834662, 'max_iter': 4612, 'tol': 0.000784906091545598}
Лучшая метрика (root_mean_squared_error): 4.3294
```

Out[15]:
```
ElasticNet
ElasticNet(alpha=0.0003546890980077183, l1_ratio=0.79903868834662,
           max_iter=4612, tol=0.000784906091545598)
```

In [16]:
```python
y_pred = best_model.predict(X_test)
```

In [17]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.195
Model score for metric "root_mean_squared_error": 3.1948
Model score for metric "r2_score": 0.6803
```



Своя модель

In [ ]:
```python
class MyElasticNet:
    """
    Parameters
    ----------
    alpha : float, default=1.0
    l1_ratio : float, default=0.5
    max_iter : int, default=1000
    tol : float, default=1e-4
    """
    def __init__(self, alpha: float = 1.0, l1_ratio: float = 0.5, max_iter: int = 1000, tol: float = 1e-4):
        self.alpha = alpha
        self.l1_ratio = l1_ratio
        self.max_iter = max_iter
        self.tol = tol

        self.coef_ = None
        self.intercept_ = None
        self._X_mean = None
        self._X_std = None
        self._y_mean = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> 'MyElasticNet':
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)

        self._X_mean = np.mean(X, axis=0)
        self._X_std = np.std(X, axis=0)
        self._X_std[self._X_std == 0] = 1
        X_std = (X - self._X_mean) / self._X_std

        self._y_mean = np.mean(y)
        y_centered = y - self._y_mean

        n_samples, n_features = X_std.shape

        w = np.zeros(n_features)

        for _ in range(self.max_iter):
            w_old = w.copy()
            for j in range(n_features):
                residual = y_centered - (X_std @ w) + X_std[:, j] * w[j]
                rho_j = (1 / n_samples) * np.dot(X_std[:, j], residual)

                l1_penalty = self.alpha * self.l1_ratio
                if rho_j < -l1_penalty:
                    w_j_update = rho_j + l1_penalty
                elif rho_j > l1_penalty:
                    w_j_update = rho_j - l1_penalty
                else:
                    w_j_update = 0

                l2_penalty = self.alpha * (1 - self.l1_ratio)
                w[j] = w_j_update / (1 + l2_penalty)

            if np.linalg.norm(w - w_old) < self.tol:
                break

        self.coef_ = w / self._X_std
        self.intercept_ = self._y_mean - np.dot(self._X_mean, self.coef_)

        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        if self.coef_ is None:
            raise RuntimeError("You must call fit() before predict().")
        X = np.asarray(X, dtype=float)
        return np.dot(X, self.coef_) + self.intercept_

    def get_params(self, deep: bool = True) -> dict:
        return {
            'alpha': self.alpha,
            'l1_ratio': self.l1_ratio,
            'max_iter': self.max_iter,
            'tol': self.tol,
        }

    def set_params(self, **params):
        for key, value in params.items():
            if hasattr(self, key):
                setattr(self, key, value)
            else:
                raise ValueError(f"Invalid parameter {key} for MyElasticNet")
        return self
```

In [19]:
```python
model_class = MyElasticNet
```

Бейзлайн

In [ ]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_raw, df_target_raw, test_size = test_size, random_state = random_state)

y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

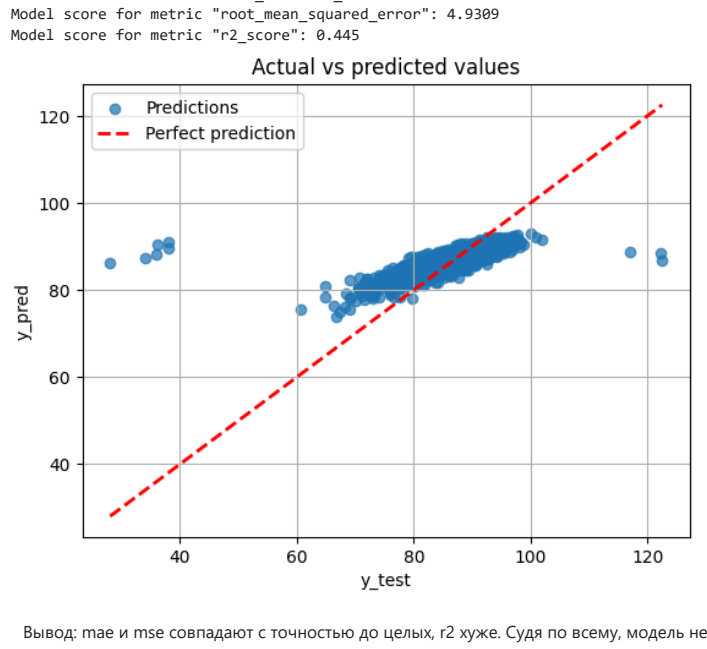
In [21]:
```python
model = model_class()

model.fit(X_train, y_train)
```

```
Out[21]: <__main__.MyElasticNet at 0x288c7f4c190>
```

In [22]:
```python
y_pred = model.predict(X_test)
```

In [23]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.9616
Model score for metric "root_mean_squared_error": 4.9309
Model score for metric "r2_score": 0.445
```



Вывод: mae и mse совпадают с точностью до целых, r2 хуже. Судя по всему, модель не успела обучиться.

Улучшенный бейзлайн

Улучшение за счёт препроцессинга

In [ ]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_processed, df_target_processed, test_size = test_size, random_state = random_state)

y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

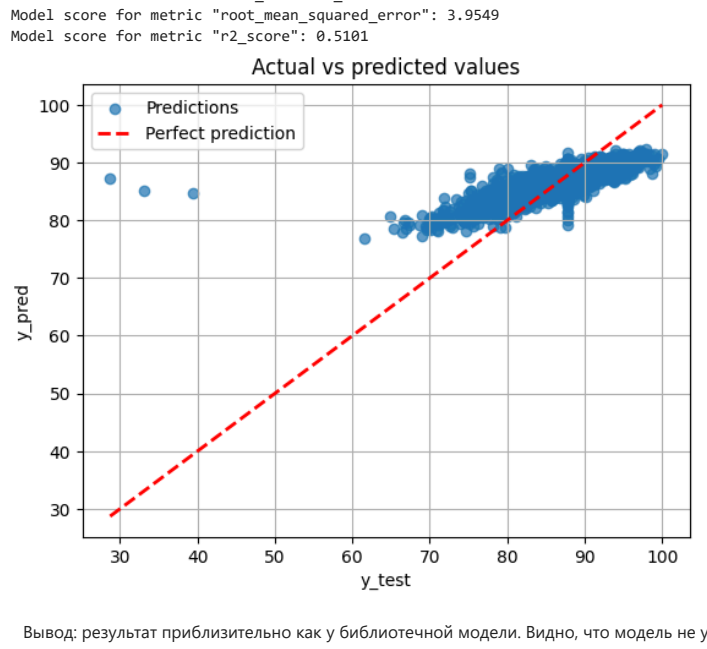
In [25]:
```python
model = model_class()

model.fit(X_train, y_train)
```

```
Out[25]: <__main__.MyElasticNet at 0x288c81d1310>
```

In [26]:
```python
y_pred = model.predict(X_test)
```

In [27]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.9091
Model score for metric "root_mean_squared_error": 3.9549
Model score for metric "r2_score": 0.5101
```



Вывод: результат приблизительно как у библиотечной модели. Видно, что модель не успела обучиться.

Дополнительное улучшение за счёт подбора параметров на cross-val

In [28]:
```python
def objective(trial):
    optuna_params = {
        'alpha': trial.suggest_float('alpha', 0.0, 1.0),
        'l1_ratio': trial.suggest_float('l1_ratio', 0.0, 1.0),
        'max_iter': trial.suggest_int('max_iter', 1000, 5000),
        'tol': trial.suggest_float('tol', 1e-5, 1e-3),
    }

    trial_model = model_class(**optuna_params)

    score = cross_val_score(trial_model, X_train, y_train, cv=cv, scoring=make_scorer(optuna_metric), n_jobs=-1).mean()

    return score

study = optuna.create_study(direction=optuna_direction, sampler=optuna.samplers.TPESampler(seed=random_state))
study.optimize(objective, n_trials=n_trials)

print(f"Лучшие параметры для модели: {study.best_params}")
print(f"Лучшая метрика ({optuna_metric.__name__}): {round(study.best_value,round_num)}")

best_model = model_class(**study.best_params)
best_model.fit(X_train, y_train)
```

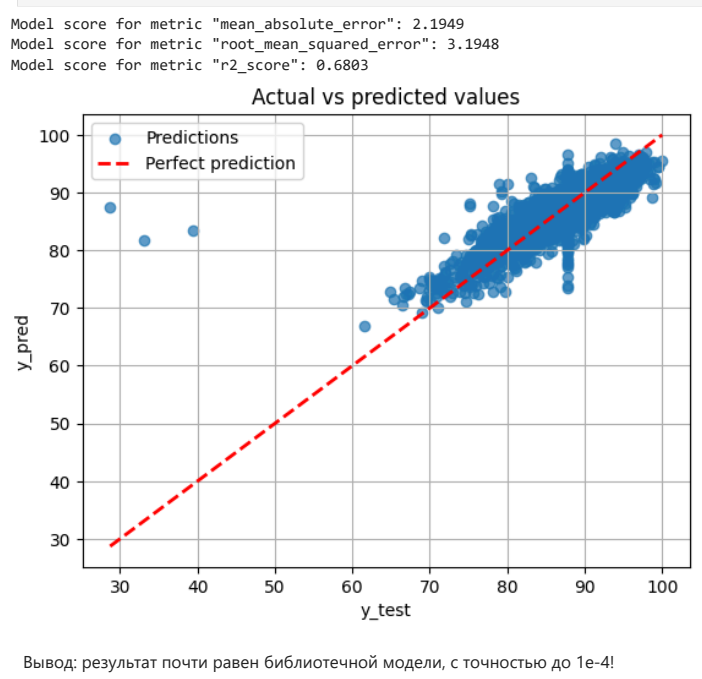
```
[I 2025-12-08 22:52:44,834] A new study created in memory with name: no-name-9797f1ae-b131-4e03-8c6c-0526afca3656
[I 2025-12-08 22:52:44,871] Trial 0 finished with value: 4.4177008023943545 and parameters: {'alpha': 0.3745401188473625, 'l1_ratio': 0.9507143064099162, 'max_iter': 3928, 'tol': 0.0006026718993550663}. Best is trial 0 with value: 4.4177008023943545.
[I 2025-12-08 22:52:44,903] Trial 1 finished with value: 4.36927246049113 and parameters: {'alpha': 0.15601864044243652, 'l1_ratio': 0.15599452033620265, 'max_iter': 1232, 'tol': 0.0008675143843171858}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:44,935] Trial 2 finished with value: 4.572152601702635 and parameters: {'alpha': 0.6011150117432088, 'l1_ratio': 0.7080725777960455, 'max_iter': 1082, 'tol': 0.0009702107536403744}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:44,966] Trial 3 finished with value: 4.787590530786359 and parameters: {'alpha': 0.8324426408004217, 'l1_ratio': 0.21233911067827616, 'max_iter': 1727, 'tol': 0.00019157046475489948}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:44,998] Trial 4 finished with value: 4.418447538684548 and parameters: {'alpha': 0.3042422429595377, 'l1_ratio': 0.5247564316322378, 'max_iter': 2728, 'tol': 0.0002983168487960615}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:45,030] Trial 5 finished with value: 4.654074686895195 and parameters: {'alpha': 0.6118528947223795, 'l1_ratio': 0.13949386065204183, 'max_iter': 2168, 'tol': 0.00037269822486075483}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:45,062] Trial 6 finished with value: 4.473835937879142 and parameters: {'alpha': 0.45606998421703593, 'l1_ratio': 0.7851759613930136, 'max_iter': 1798, 'tol': 0.0005190920940294755}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:45,094] Trial 7 finished with value: 4.653019993945786 and parameters: {'alpha': 0.5924145688620425, 'l1_ratio': 0.046450412719997725, 'max_iter': 3430, 'tol': 0.0001788188824504186}. Best is trial 1 with value: 4.36927246049113.
[I 2025-12-08 22:52:45,126] Trial 8 finished with value: 4.332572624446621 and parameters: {'alpha': 0.06505159298527952, 'l1_ratio': 0.9488855372533332, 'max_iter': 4863, 'tol': 0.0008103133746352965}. Best is trial 8 with value: 4.332572624446621.
[I 2025-12-08 22:52:45,158] Trial 9 finished with value: 4.454754860571947 and parameters: {'alpha': 0.3046137691733707, 'l1_ratio': 0.09767211400638387, 'max_iter': 3737, 'tol': 0.0004457509688022053}. Best is trial 8 with value: 4.332572624446621.
[I 2025-12-08 22:52:45,207] Trial 10 finished with value: 4.3301316148674225 and parameters: {'alpha': 0.020546809824298393, 'l1_ratio': 0.47332732145466855, 'max_iter': 4879, 'tol': 0.0007125434479173731}. Best is trial 10 with value: 4.3301316148674
225.
[I 2025-12-08 22:52:45,236] Trial 11 finished with value: 4.329842312609182 and parameters: {'alpha': 0.01454087516236438, 'l1_ratio': 0.4016647941567012, 'max_iter': 4949, 'tol': 0.0007634576416083034}. Best is trial 11 with value: 4.329842312609182.
[I 2025-12-08 22:52:45,267] Trial 12 finished with value: 4.331414978428629 and parameters: {'alpha': 0.03435746358874557, 'l1_ratio': 0.3796042434202235, 'max_iter': 4984, 'tol': 0.0006826975469036963}. Best is trial 11 with value: 4.329842312609182.
[I 2025-12-08 22:52:45,298] Trial 13 finished with value: 4.367414343202278 and parameters: {'alpha': 0.17550991593332854, 'l1_ratio': 0.41508030158786696, 'max_iter': 4432, 'tol': 0.0007283047566480887}. Best is trial 11 with value: 4.32984231260918
2.
[I 2025-12-08 22:52:45,330] Trial 14 finished with value: 4.3296374536548345 and parameters: {'alpha': 0.010813209728300745, 'l1_ratio': 0.5760103475555451, 'max_iter': 4330, 'tol': 0.000937733682004973}. Best is trial 14 with value: 4.329637453654834
5.
[I 2025-12-08 22:52:45,362] Trial 15 finished with value: 4.796600781692016 and parameters: {'alpha': 0.9119167232179372, 'l1_ratio': 0.6895366011206421, 'max_iter': 4327, 'tol': 0.0009500202567910115}. Best is trial 14 with value: 4.3296374536548345.
[I 2025-12-08 22:52:45,408] Trial 16 finished with value: 4.375328736121665 and parameters: {'alpha': 0.1830610799292295, 'l1_ratio': 0.29974150227632423, 'max_iter': 3068, 'tol': 0.0008267541706232683}. Best is trial 14 with value: 4.329637453654834
5.
[I 2025-12-08 22:52:45,440] Trial 17 finished with value: 4.388114934276159 and parameters: {'alpha': 0.24387496086356825, 'l1_ratio': 0.5586366447622302, 'max_iter': 4276, 'tol': 0.0009830672492423369}. Best is trial 14 with value: 4.329637453654834
5.
[I 2025-12-08 22:52:45,471] Trial 18 finished with value: 4.338703069867511 and parameters: {'alpha': 0.09039590229092494, 'l1_ratio': 0.5864590448357907, 'max_iter': 3946, 'tol': 0.0005542483582499034}. Best is trial 14 with value: 4.329637453654834
5.
[I 2025-12-08 22:52:45,503] Trial 19 finished with value: 4.71072341033941 and parameters: {'alpha': 0.7298348295317014, 'l1_ratio': 0.30344630088855795, 'max_iter': 4638, 'tol': 0.0008808177874592472}. Best is trial 14 with value: 4.3296374536548345.
[I 2025-12-08 22:52:45,534] Trial 20 finished with value: 4.447819407053114 and parameters: {'alpha': 0.4134140996927128, 'l1_ratio': 0.8152590804011588, 'max_iter': 3235, 'tol': 0.0006332449750962679}. Best is trial 14 with value: 4.3296374536548345.
[I 2025-12-08 22:52:45,566] Trial 21 finished with value: 4.331556216810079 and parameters: {'alpha': 0.03720314284198487, 'l1_ratio': 0.4456913325009604, 'max_iter': 4995, 'tol': 0.0007508122756340961}. Best is trial 14 with value: 4.329637453654834
5.
[I 2025-12-08 22:52:45,597] Trial 22 finished with value: 4.329629827768617 and parameters: {'alpha': 0.010989189243771236, 'l1_ratio': 0.6393870521051082, 'max_iter': 4568, 'tol': 0.0007492947161424528}. Best is trial 22 with value: 4.32962982776861
7.
[I 2025-12-08 22:52:45,628] Trial 23 finished with value: 4.349353226363565 and parameters: {'alpha': 0.14158424165235128, 'l1_ratio': 0.6423620561394953, 'max_iter': 4086, 'tol': 3.070443081856989e-05}. Best is trial 22 with value: 4.329629827768617.
[I 2025-12-08 22:52:45,660] Trial 24 finished with value: 4.329427636220271 and parameters: {'alpha': 0.0003546890980077183, 'l1_ratio': 0.79903868834662, 'max_iter': 4612, 'tol': 0.000784906091545598}. Best is trial 24 with value: 4.329427636220271.
[I 2025-12-08 22:52:45,692] Trial 25 finished with value: 4.380686116475762 and parameters: {'alpha': 0.2661907052857681, 'l1_ratio': 0.8420867358525691, 'max_iter': 3666, 'tol': 0.0008923922551398093}. Best is trial 24 with value: 4.329427636220271.
[I 2025-12-08 22:52:45,723] Trial 26 finished with value: 4.342402827930179 and parameters: {'alpha': 0.11994792988037481, 'l1_ratio': 0.7394147599861284, 'max_iter': 4518, 'tol': 0.0006437772418893812}. Best is trial 24 with value: 4.329427636220271.
[I 2025-12-08 22:52:45,754] Trial 27 finished with value: 4.377780294250224 and parameters: {'alpha': 0.22768745673289145, 'l1_ratio': 0.6274198266149676, 'max_iter': 4624, 'tol': 0.0007995717608725975}. Best is trial 24 with value: 4.329427636220271.
[I 2025-12-08 22:52:45,786] Trial 28 finished with value: 4.3381192298578535 and parameters: {'alpha': 0.09222316232528953, 'l1_ratio': 0.6677924495840926, 'max_iter': 2508, 'tol': 0.0009050689171865545}. Best is trial 24 with value: 4.32942763622027
1.
[I 2025-12-08 22:52:45,817] Trial 29 finished with value: 4.418161165644243 and parameters: {'alpha': 0.3682458851596138, 'l1_ratio': 0.9057271321920898, 'max_iter': 4010, 'tol': 0.000590567392374896}. Best is trial 24 with value: 4.329427636220271.
Лучшие параметры для модели: {'alpha': 0.0003546890980077183, 'l1_ratio': 0.79903868834662, 'max_iter': 4612, 'tol': 0.000784906091545598}
Лучшая метрика (root_mean_squared_error): 4.3294
```

```
Out[28]: <__main__.MyElasticNet at 0x288c813b850>
```

In [29]:
```python
y_pred = best_model.predict(X_test)
```

In [30]:
```python
evaluate(metrics=eval_metrics, y_test=y_test, y_pred=y_pred, round_num=round_num)
```

```
Model score for metric "mean_absolute_error": 2.1949
Model score for metric "root_mean_squared_error": 3.1948
Model score for metric "r2_score": 0.6803
```



Вывод: результат почти равен библиотечной модели, с точностью до 1e-4!

Классификация

In [31]:
```python
RAW_FEATS = 'classification_raw_feats.csv'
RAW_TARGET = 'classification_raw_target.csv'
PROCESSED_FEATS = 'classification_processed_feats.csv'
PROCESSED_TARGET = 'classification_processed_target.csv'
```

In [32]:
```python
df_feats_raw = pd.read_csv(RAW_FEATS)
df_target_raw = pd.read_csv(RAW_TARGET)
df_feats_processed = pd.read_csv(PROCESSED_FEATS)
df_target_processed = pd.read_csv(PROCESSED_TARGET)
```

In [33]:
```python
def evaluate(y_test, y_pred_class, y_pred_proba, round_num):

    fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:,1])
    roc_auc = roc_auc_score(y_test, y_pred_proba[:,1])

    print(f"ROC AUC: {round(roc_auc,round_num)}", '\n')

    f1 = f1_score(y_test, y_pred_class)
    precision = precision_score(y_test, y_pred_class)
    recall = recall_score(y_test, y_pred_class)

    print(f"Precision: {round(precision,round_num)}")
    print(f"Recall: {round(recall,round_num)}")
    print(f"F1-Score: {round(f1,round_num)}")

    plt.plot(fpr, tpr, label=f'ROC curve')
    plt.plot([0, 1], [0, 1], linestyle='--', label='Random classifier')
    plt.xlabel('FPR')
    plt.ylabel('TPR')
    plt.title('ROC curve')
    plt.legend(loc="lower right")
    plt.show()

    cm = confusion_matrix(y_test, y_pred_class)
    sns.heatmap(cm, annot=True, fmt = 'd', cmap='Blues', cbar_kws={'label': 'Count'})
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion matrix')
    plt.show()
```

In [34]:
```python
eval_metrics = [f1_score, roc_auc_score]
random_state = 42
test_size = 0.2

round_num = 4

optuna_metric = f1_score
optuna_direction = "maximize"
n_trials = 20
cv = 5
```

Готовая модель

In [35]:
```python
model_class = LogisticRegression
```

Бейзлайн

In [36]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_raw, df_target_raw, test_size = test_size, random_state = random_state)
```

In [37]:
```python
y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

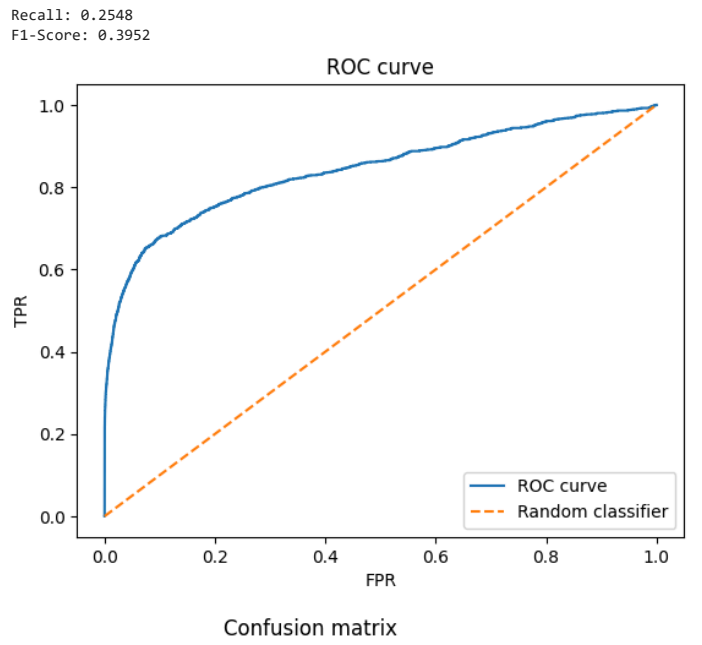
In [38]:
```python
model = model_class(solver='liblinear')

model.fit(X_train, y_train)
```

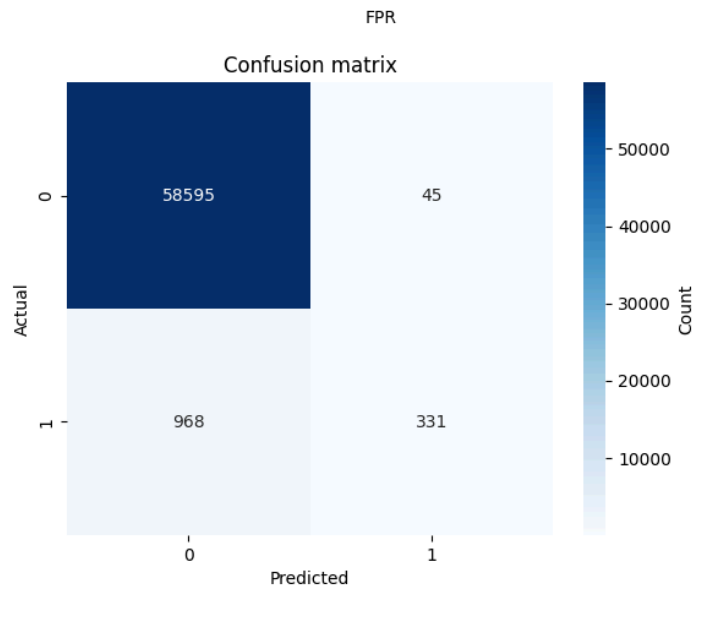
Out[38]:
```
LogisticRegression
LogisticRegression(solver='liblinear')
```

In [39]:
```python
y_pred_class = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test)
```

In [40]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.8426

Precision: 0.8803
Recall: 0.2548
F1-Score: 0.3952
```





Улучшенный бейзлайн

Улучшение за счёт препроцессинга

In [41]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_processed, df_target_processed, test_size = test_size, random_state = random_state)
```

In [42]:
```python
y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

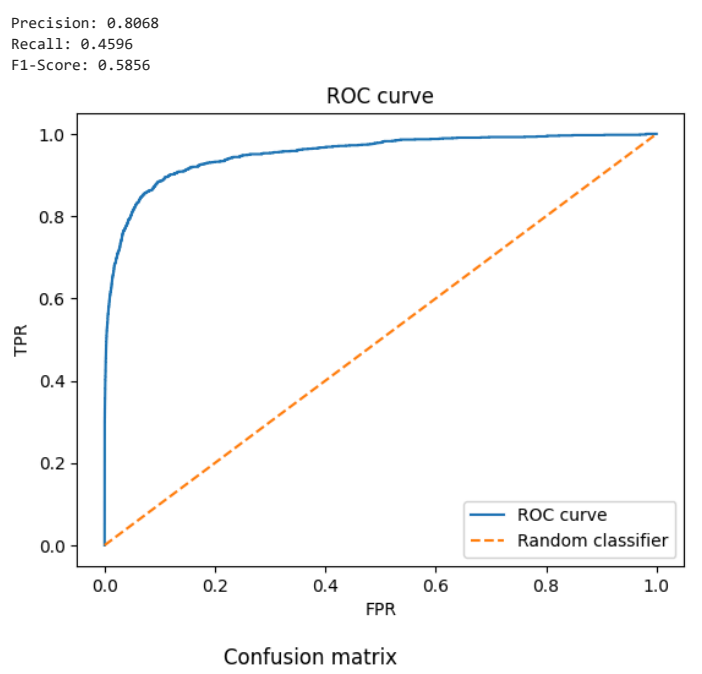
In [43]:
```python
model = model_class(solver='liblinear')

model.fit(X_train, y_train)
```

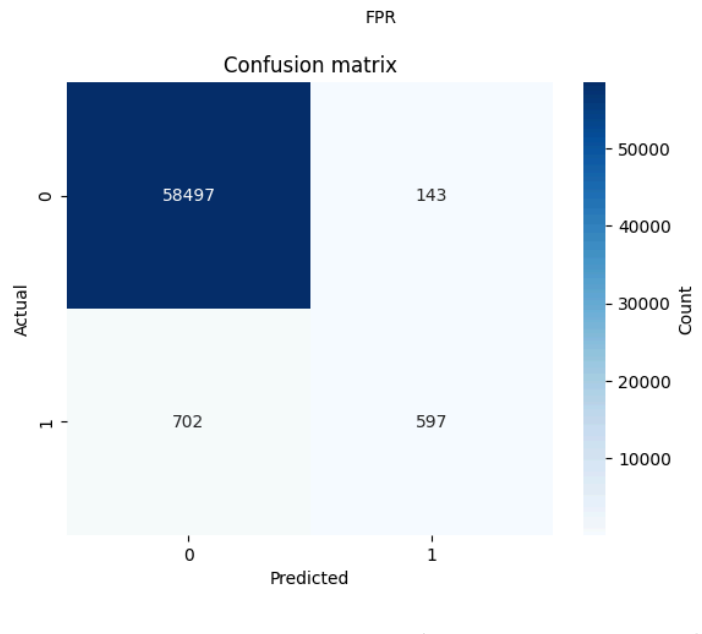
Out[43]:
```
LogisticRegression
LogisticRegression(solver='liblinear')
```

In [44]:
```python
y_pred_class = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test)
```

In [45]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.9524

Precision: 0.8068
Recall: 0.4596
F1-Score: 0.5856
```





Дополнительное улучшение за счёт подбора параметров на cross-val

In [46]:
```python
def objective(trial):
    optuna_params = {
        'solver': trial.suggest_categorical('solver', ['liblinear']),
        'fit_intercept': trial.suggest_categorical('fit_intercept', [True, False]),
        'intercept_scaling': trial.suggest_float('intercept_scaling', 0.1, 10.0),
        'max_iter': trial.suggest_int('max_iter', 100, 2000),
        'C': trial.suggest_float('C', 1e-4, 1e4),
        'penalty': trial.suggest_categorical('penalty', ['l1', 'l2']),
        'tol': trial.suggest_float('tol', 1e-5, 1e-3),
    }

    trial_model = model_class(**optuna_params)

    score = cross_val_score(trial_model, X_train, y_train, cv=cv, scoring=make_scorer(optuna_metric), n_jobs=-1).mean()

    return score

study = optuna.create_study(direction=optuna_direction, sampler=optuna.samplers.TPESampler(seed=random_state))
study.optimize(objective, n_trials=n_trials)

print(f"Лучшие параметры для модели: {study.best_params}")
print(f"Лучшая метрика ({optuna_metric.__name__}): {round(study.best_value,round_num)}")

best_model = model_class(**study.best_params)
best_model.fit(X_train, y_train)
```

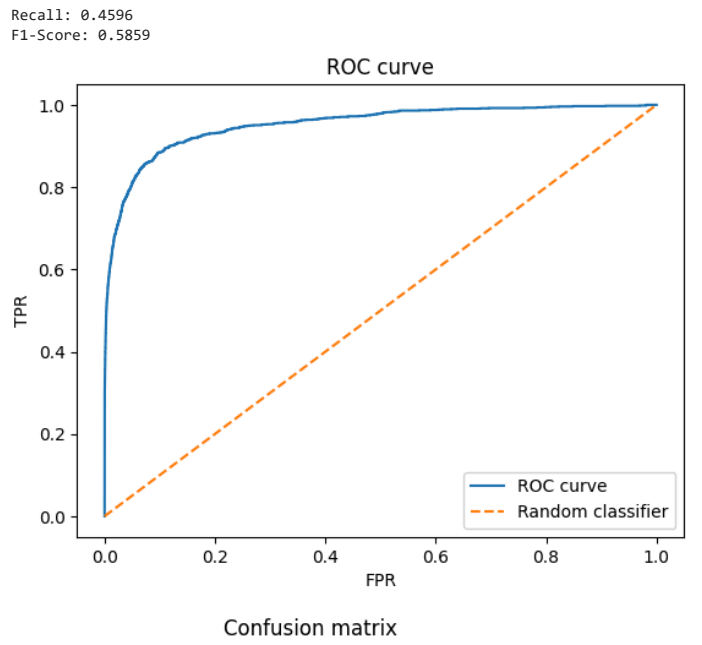
```
[I 2025-12-08 22:52:49,553] A new study created in memory with name: no-name-69bbe7d5-b440-462f-920d-2884e6d369df
[I 2025-12-08 22:52:50,476] Trial 0 finished with value: 0.5486080214954976 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 7.34674002393291, 'max_iter': 1238, 'C': 1560.1864888225011, 'penalty': 'l1', 'tol': 0.000
8675143843171858}. Best is trial 0 with value: 0.5486080214954976.
[I 2025-12-08 22:52:51,553] Trial 1 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 0.3037864935284442, 'max_iter': 1943, 'C': 8324.426424759953, 'penalty': 'l1', 'tol': 0.00
019157046475489948}. Best is trial 1 with value: 0.5490115087402506.
[I 2025-12-08 22:52:52,437] Trial 2 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 4.376255684556946, 'max_iter': 653, 'C': 6118.5289860385055, 'penalty': 'l2', 'tol': 0.000
37269822486075483}. Best is trial 1 with value: 0.5490115087402506.
[I 2025-12-08 22:52:53,350] Trial 3 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 2.0767704433677614, 'max_iter': 1077, 'C': 5924.145729378968, 'penalty': 'l2', 'tol': 0.00
01788188824504186}. Best is trial 1 with value: 0.5490115087402506.
[I 2025-12-08 22:52:54,246] Trial 4 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 9.659757127438137, 'max_iter': 1636, 'C': 3046.1377612723304, 'penalty': 'l2', 'tol': 0.00
04457509688022053}. Best is trial 1 with value: 0.5490115087402506.
[I 2025-12-08 22:52:55,335] Trial 5 finished with value: 0.5488430521390985 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 0.4404463590406621, 'max_iter': 1828, 'C': 2587.7998901221713, 'penalty': 'l1', 'tol': 0.0
005248673409660327}. Best is trial 1 with value: 0.5490115087402506.
[I 2025-12-08 22:52:59,739] Trial 6 finished with value: 0.5937962180422016 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 9.69888781486913, 'max_iter': 1573, 'C': 9394.989421691997, 'penalty': 'l1', 'tol': 0.00092
26554926728857}. Best is trial 6 with value: 0.5937962180422016.
[I 2025-12-08 22:53:00,668] Trial 7 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 0.5477501602143269, 'max_iter': 718, 'C': 3886.7729580270916, 'penalty': 'l2', 'tol': 0.00
03631857934266534}. Best is trial 6 with value: 0.5937962180422016.
[I 2025-12-08 22:53:01,742] Trial 8 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 1.4951498272501504, 'max_iter': 1624, 'C': 745.5065293426438, 'penalty': 'l1', 'tol': 0.000
20672825471883069}. Best is trial 6 with value: 0.5937962180422016.
[I 2025-12-08 22:53:02,631] Trial 9 finished with value: 0.5490115087402506 and parameters: {'solver': 'liblinear', 'fit_intercept': False, 'intercept_scaling': 7.097887704091409, 'max_iter': 1485, 'C': 7712.703489732424, 'penalty': 'l2', 'tol': 0.000
12471036892987842}. Best is trial 6 with value: 0.5937962180422016.
[I 2025-12-08 22:53:07,118] Trial 10 finished with value: 0.5938989139951888 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 9.3339130502402, 'max_iter': 114, 'C': 9551.399374250495, 'penalty': 'l1', 'tol': 0.000908
5881473296926}. Best is trial 10 with value: 0.5938989139951888.
[I 2025-12-08 22:53:11,719] Trial 11 finished with value: 0.5938989139951888 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 9.867359240709137, 'max_iter': 263, 'C': 9974.434860276902, 'penalty': 'l1', 'tol': 0.0009
463516227739758}. Best is trial 10 with value: 0.5938989139951888.
[I 2025-12-08 22:53:16,382] Trial 12 finished with value: 0.5939699202021107 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 7.893821273897779, 'max_iter': 158, 'C': 9704.052171353387, 'penalty': 'l1', 'tol': 0.0007
699497377237833}. Best is trial 12 with value: 0.5939699202021107.
[I 2025-12-08 22:53:21,117] Trial 13 finished with value: 0.5941434087062264 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 7.694534641062876, 'max_iter': 183, 'C': 7661.630609661064, 'penalty': 'l1', 'tol': 0.0007
033827383029576}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:25,418] Trial 14 finished with value: 0.5938989139951888 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 6.733544191032303, 'max_iter': 491, 'C': 7119.778050936338, 'penalty': 'l1', 'tol': 0.0007
189217923076215}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:30,113] Trial 15 finished with value: 0.5938007714615333 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 5.409977017954469, 'max_iter': 385, 'C': 8360.389655656763, 'penalty': 'l1', 'tol': 0.0006
886495620476449}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:35,336] Trial 16 finished with value: 0.5940724024993045 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 8.341894406637886, 'max_iter': 812, 'C': 6405.020815809868, 'penalty': 'l1', 'tol': 0.0007
086240457579481}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:39,923] Trial 17 finished with value: 0.5939699202021107 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 5.132662711493888, 'max_iter': 805, 'C': 4675.728862797467, 'penalty': 'l1', 'tol': 0.0005
929485917856858}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:45,039] Trial 18 finished with value: 0.5939699202021107 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 8.517586823007008, 'max_iter': 946, 'C': 6424.846845713453, 'penalty': 'l1', 'tol': 0.0006
145839174949879}. Best is trial 13 with value: 0.5941434087062264.
[I 2025-12-08 22:53:49,585] Trial 19 finished with value: 0.5939699202021107 and parameters: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 6.392459028593608, 'max_iter': 528, 'C': 5310.380662815907, 'penalty': 'l1', 'tol': 0.0008
296671944713892}. Best is trial 13 with value: 0.5941434087062264.
Лучшие параметры для модели: {'solver': 'liblinear', 'fit_intercept': True, 'intercept_scaling': 7.694534641062876, 'max_iter': 183, 'C': 7661.630609661064, 'penalty': 'l1', 'tol': 0.0007033827383029576}
Лучшая метрика (f1_score): 0.5941
```

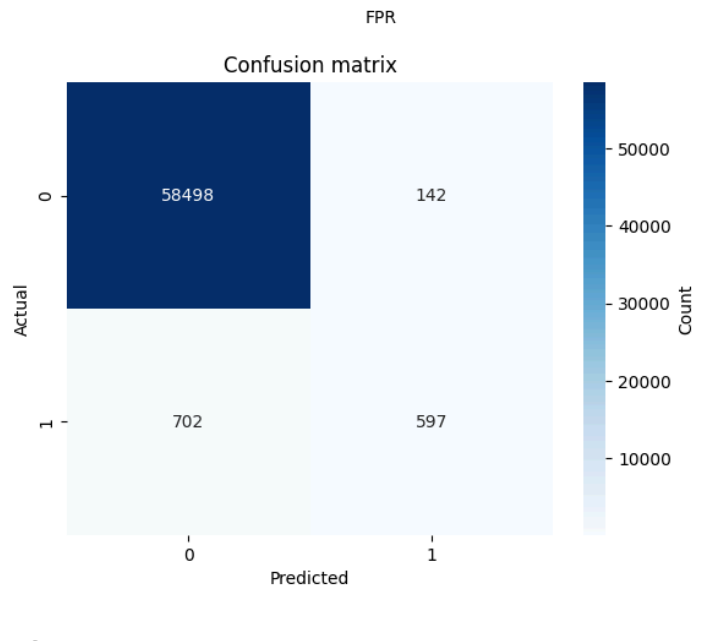
Out[46]:
```
LogisticRegression
LogisticRegression(C=7661.630609661064, intercept_scaling=7.694534641062876,
                   max_iter=183, penalty='l1', solver='liblinear',
                   tol=0.0007033827383029576)
```

In [47]:
```python
y_pred_class = best_model.predict(X_test)
y_pred_proba = best_model.predict_proba(X_test)
```

In [48]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.9523

Precision: 0.8078
Recall: 0.4596
F1-Score: 0.5859
```





Своя модель

In [ ]:
```python
class MyLogisticRegression:
    """
    Parameters
    ----------
    fit_intercept : bool, default=True
    intercept_scaling : float, default=1.0
    max_iter : int, default=1000
    C : float, default=1.0
    penalty : {'l1', 'l2'}, default='l2'
    tol : float, default=1e-4
    threshold : float, devault = 0.5
    """
    def __init__(self, fit_intercept: bool = True, intercept_scaling: float = 1.0,
                 max_iter: int = 1000, C: float = 1.0, penalty: str = 'l2',
                 tol: float = 1e-4, threshold: float = 0.5):
        self.fit_intercept = fit_intercept
        self.intercept_scaling = intercept_scaling
        self.max_iter = max_iter
        self.C = C
        self.penalty = penalty
        self.tol = tol
        self.threshold = threshold

        self.coef_ = None
        self.intercept_ = 0.0
        self._X_mean = None
        self._X_std = None
        self._n_features = None

    def _sigmoid(self, z: np.ndarray) -> np.ndarray:
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X: np.ndarray, y: np.ndarray) -> 'MyLogisticRegression':
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)

        if X.shape[0] != y.shape[0]:
            raise ValueError("Number of samples in X and y must be equal.")

        n_samples, self._n_features = X.shape

        self._X_mean = np.mean(X, axis=0)
        self._X_std = np.std(X, axis=0)
        self._X_std[self._X_std == 0] = 1
        X_std = (X - self._X_mean) / self._X_std

        if self.fit_intercept:
            synthetic_feature = np.full((n_samples, 1), self.intercept_scaling)
            X_std = np.hstack([X_std, synthetic_feature])

        w = np.zeros(X_std.shape[1])
        lambda_ = 1.0 / self.C

        for _ in range(self.max_iter):
            w_old = w.copy()

            for j in range(len(w)):
                z = X_std @ w
                p = self._sigmoid(z)
                residuals = p - y

                grad_j = (1 / n_samples) * np.dot(X_std[:, j], residuals)

                w_j_approx = w[j] - grad_j

                if self.penalty == 'l1':
                    if w_j_approx > 0:
                        w[j] = max(0, w_j_approx - lambda_)
                    else:
                        w[j] = min(0, w_j_approx + lambda_)
                elif self.penalty == 'l2':
                    w[j] = w_j_approx / (1 + lambda_)

            if np.sum(np.abs(w - w_old)) < self.tol:
                break

        if self.fit_intercept:
            self.coef_ = w[:-1] / self._X_std
            self.intercept_ = w[-1] * self.intercept_scaling
            self.intercept_ -= np.dot(self._X_mean, self.coef_)
        else:
            self.coef_ = w / self._X_std
            self.intercept_ = 0.0

        return self

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        if self.coef_ is None:
            raise RuntimeError("You must call fit() before predict_proba().")
        X = np.asarray(X, dtype=float)

        z = X @ self.coef_ + self.intercept_
        proba_1 = self._sigmoid(z)

        proba_0 = 1 - proba_1
        return np.vstack([proba_0, proba_1]).T

    def predict(self, X: np.ndarray) -> np.ndarray:
        proba = self.predict_proba(X)
        return (proba[:, 1] >= self.threshold).astype(int)

    def get_params(self, deep: bool = True) -> dict:
        return {
            'fit_intercept': self.fit_intercept,
            'intercept_scaling': self.intercept_scaling,
            'max_iter': self.max_iter,
            'C': self.C,
            'penalty': self.penalty,
            'tol': self.tol,
        }

    def set_params(self, **params):
        for key, value in params.items():
            if hasattr(self, key):
                setattr(self, key, value)
            else:
                raise ValueError(f"Invalid parameter {key} for MyLogisticRegression")
        return self
```

In [50]:
```python
model_class = MyLogisticRegression
```

Бейзлайн

In [51]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_raw, df_target_raw, test_size = test_size, random_state = random_state)
```

In [52]:
```python
y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

In [53]:
```python
model = model_class()

model.fit(X_train, y_train)
```

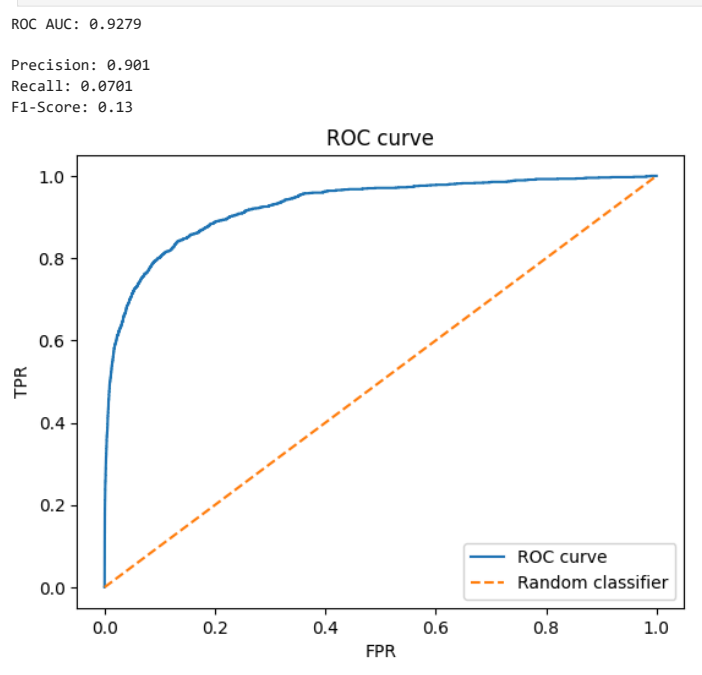
```
Out[53]: <__main__.MyLogisticRegression at 0x288c946ab80>
```

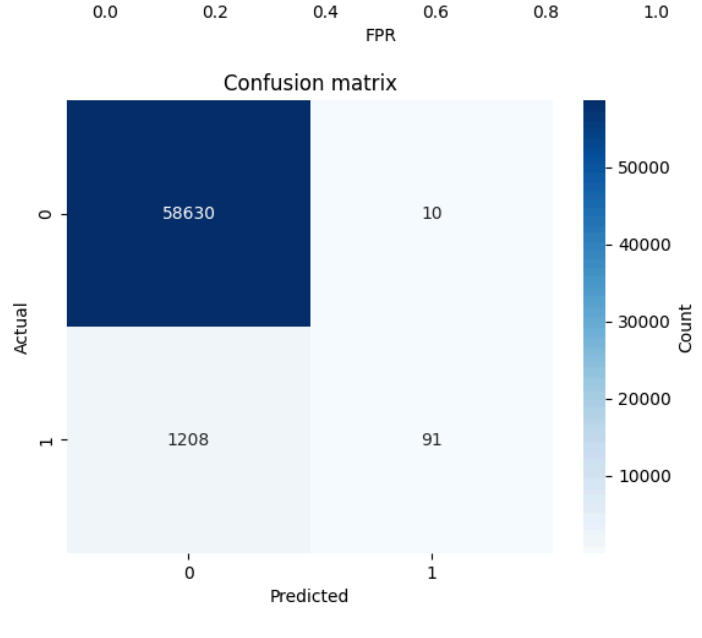
In [54]:
```python
y_pred_class = model.predict(X_test)
```

In [55]:
```python
y_pred_proba = model.predict_proba(X_test)
```

In [56]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.9279

Precision: 0.901
Recall: 0.0701
F1-Score: 0.13
```





Высокий ROC-AUC, но F1 сильно хуже в сравнении с библиотечной моделью.

Улучшенный бейзлайн

Улучшение за счёт препроцессинга

In [57]:
```python
X_train, X_test, y_train, y_test = train_test_split(df_feats_processed, df_target_processed, test_size = test_size, random_state = random_state)
```

In [58]:
```python
y_train = y_train.values.ravel()
y_test = y_test.values.ravel()
```

In [59]:
```python
model = model_class()

model.fit(X_train, y_train)
```

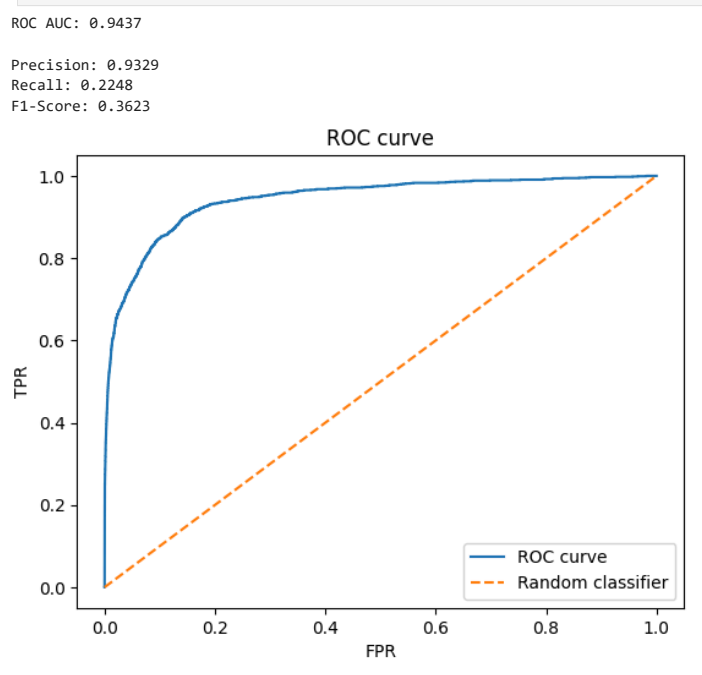
```
Out[59]: <__main__.MyLogisticRegression at 0x288c7fb2220>
```

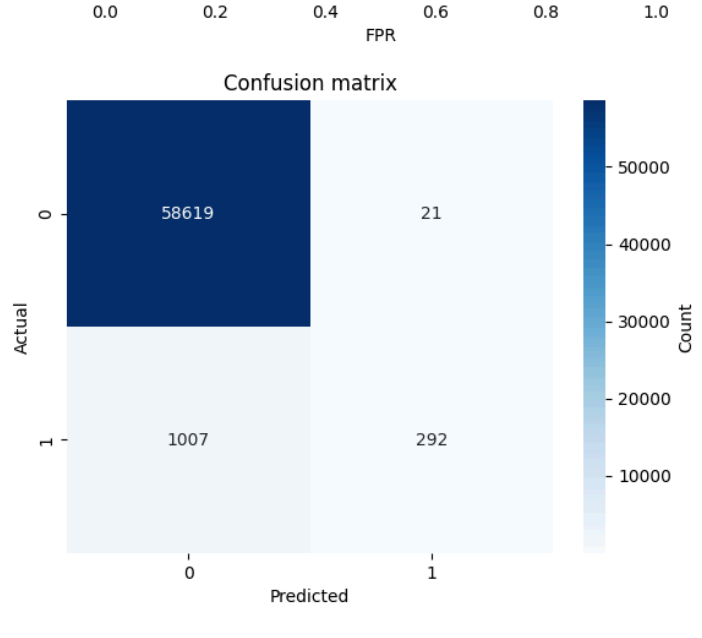
In [60]:
```python
y_pred_class = model.predict(X_test)
```

In [61]:
```python
y_pred_proba = model.predict_proba(X_test)
```

In [62]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.9437

Precision: 0.9329
Recall: 0.2248
F1-Score: 0.3623
```





F1 улучшился, но всё ещё сильно хуже чем у библиотечной модели.

Дополнительное улучшение за счёт подбора параметров на cross-val

In [63]:
```python
def objective(trial):
    optuna_params = {
        'fit_intercept': trial.suggest_categorical('fit_intercept', [True, False]),
        'intercept_scaling': trial.suggest_float('intercept_scaling', 0.1, 10.0),
        'max_iter': trial.suggest_int('max_iter', 100, 2000),
        'C': trial.suggest_float('C', 1e-4, 1e4),
        'penalty': trial.suggest_categorical('penalty', ['l1', 'l2']),
        'tol': trial.suggest_float('tol', 1e-5, 1e-3),
        'threshold': trial.suggest_float('threshold', 0.0, 1.0),
    }

    trial_model = model_class(**optuna_params)

    score = cross_val_score(trial_model, X_train, y_train, cv=cv, scoring=make_scorer(optuna_metric), n_jobs=-1).mean()

    return score

study = optuna.create_study(direction=optuna_direction, sampler=optuna.samplers.TPESampler(seed=random_state))
study.optimize(objective, n_trials=n_trials)

print(f"Лучшие параметры для модели: {study.best_params}")
print(f"Лучшая метрика ({optuna_metric.__name__}): {round(study.best_value,round_num)}")

best_model = model_class(**study.best_params)
best_model.fit(X_train, y_train)
```

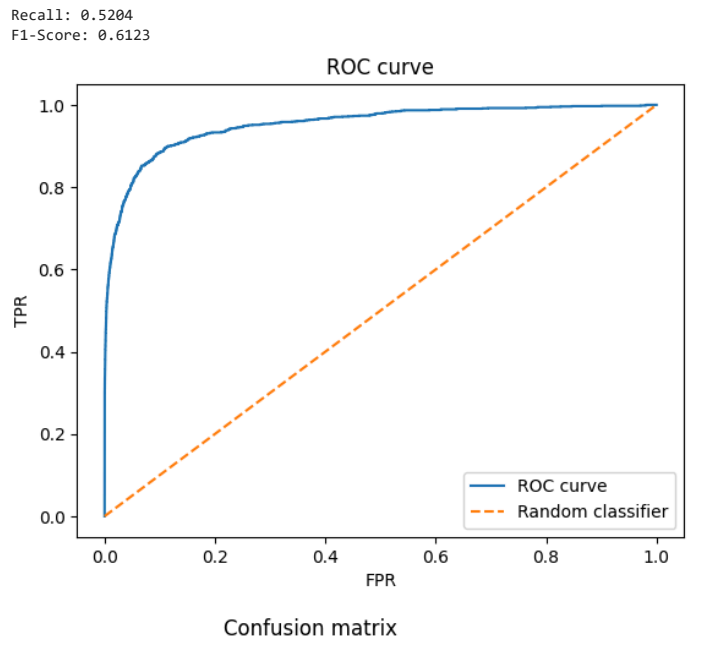
```
[I 2025-12-08 22:53:54,307] A new study created in memory with name: no-name-ce692a35-034f-4a84-8854-d2369b2606eb
[I 2025-12-08 22:54:01,860] Trial 0 finished with value: 0.130828083057096 and parameters: {'fit_intercept': False, 'intercept_scaling': 7.34674002393291, 'max_iter': 1238, 'C': 1560.1864888225011, 'penalty': 'l1', 'tol': 0.0008675143843171858, 'thres
hold': 0.6011150117432088}. Best is trial 0 with value: 0.130828083057096.
[I 2025-12-08 22:55:33,797] Trial 1 finished with value: 0.5882427611958314 and parameters: {'fit_intercept': True, 'intercept_scaling': 9.702107536403744, 'max_iter': 1682, 'C': 2123.391185548851, 'penalty': 'l2', 'tol': 0.00031119982052994234, 'thre
shold': 0.5247564316322378}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:56:21,731] Trial 2 finished with value: 0.586372793773914 and parameters: {'fit_intercept': True, 'intercept_scaling': 6.157343657751556, 'max_iter': 365, 'C': 2921.446556137717, 'penalty': 'l2', 'tol': 0.0007873242017790835, 'thresho
ld': 0.19967378215835974}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:56:29,257] Trial 3 finished with value: 0.10949211680809 and parameters: {'fit_intercept': False, 'intercept_scaling': 0.5598590859279775, 'max_iter': 1254, 'C': 1705.2413198205031, 'penalty': 'l2', 'tol': 0.0009695757127438138, 'thr
eshold': 0.8083973481164611}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:57:17,460] Trial 4 finished with value: 0.5769747894858427 and parameters: {'fit_intercept': True, 'intercept_scaling': 6.873906962470353, 'max_iter': 936, 'C': 1220.3824362439648, 'penalty': 'l1', 'tol': 0.0009102271980579943, 'thres
hold': 0.2587799816000169}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:58:11,349] Trial 5 finished with value: 0.5818885156415357 and parameters: {'fit_intercept': True, 'intercept_scaling': 5.248673409660327, 'max_iter': 1139, 'C': 1848.544636769825, 'penalty': 'l1', 'tol': 0.0009401039521485473, 'thres
hold': 0.894827354076488}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:58:21,981] Trial 6 finished with value: 0.10775483649709475 and parameters: {'fit_intercept': False, 'intercept_scaling': 0.976075770314003, 'max_iter': 472, 'C': 452.27298458265176, 'penalty': 'l2', 'tol': 0.000278635541456157, 'thre
shold': 0.8287375091519293}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:59:09,077] Trial 7 finished with value: 0.5879001686150251 and parameters: {'fit_intercept': True, 'intercept_scaling': 5.47269122326666, 'max_iter': 367, 'C': 8021.9698273207, 'penalty': 'l2', 'tol': 0.0007745223216036909, 'threshol
d': 0.1987156815341724}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 22:59:22,440] Trial 8 finished with value: 0.09624380425398234 and parameters: {'fit_intercept': False, 'intercept_scaling': 7.097887704091409, 'max_iter': 1485, 'C': 7712.703489732424, 'penalty': 'l2', 'tol': 0.00012471036892987842, 'th
reshold': 0.8631034258755935}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 23:00:49,020] Trial 9 finished with value: 0.5740829617746505 and parameters: {'fit_intercept': True, 'intercept_scaling': 0.729227667831634, 'max_iter': 691, 'C': 3251.833287749139, 'penalty': 'l1', 'tol': 0.0008883406151505634, 'thresh
old': 0.4722149251619493}. Best is trial 1 with value: 0.5882427611958314.
[I 2025-12-08 23:02:13,547] Trial 10 finished with value: 0.5910747041583357 and parameters: {'fit_intercept': True, 'intercept_scaling': 9.333881305431152, 'max_iter': 1943, 'C': 5530.093431483937, 'penalty': 'l2', 'tol': 0.0004854234295129062, 'thre
shold': 0.015594799658456338}. Best is trial 10 with value: 0.5910747041583357.
[I 2025-12-08 23:03:36,815] Trial 11 finished with value: 0.5912267161776142 and parameters: {'fit_intercept': True, 'intercept_scaling': 9.829609163696544, 'max_iter': 1996, 'C': 5269.97384395673, 'penalty': 'l2', 'tol': 0.000490103539507943, 'thresh
old': 0.0519389453390651}. Best is trial 11 with value: 0.5912267161776142.
[I 2025-12-08 23:04:56,939] Trial 12 finished with value: 0.5911527532322969 and parameters: {'fit_intercept': True, 'intercept_scaling': 9.69377908679367, 'max_iter': 1991, 'C': 5621.712222865981, 'penalty': 'l2', 'tol': 0.0005565059776747638, 'thres
hold': 0.04507622533771835}. Best is trial 11 with value: 0.5912267161776142.
[I 2025-12-08 23:06:14,924] Trial 13 finished with value: 0.590807308124216 and parameters: {'fit_intercept': True, 'intercept_scaling': 8.316150162994465, 'max_iter': 1911, 'C': 5396.201114065966, 'penalty': 'l2', 'tol': 0.0005892790438612837, 'thres
hold': 0.011116166530521589}. Best is trial 11 with value: 0.5912267161776142.
[I 2025-12-08 23:07:36,999] Trial 14 finished with value: 0.5911146773736882 and parameters: {'fit_intercept': True, 'intercept_scaling': 3.209635461843301, 'max_iter': 1639, 'C': 6868.608352314452, 'penalty': 'l2', 'tol': 0.0005705039661599788, 'thre
shold': 0.32197082779781516}. Best is trial 11 with value: 0.5912267161776142.
[I 2025-12-08 23:09:10,106] Trial 15 finished with value: 0.5913552185267388 and parameters: {'fit_intercept': True, 'intercept_scaling': 3.3760371338874675, 'max_iter': 1935, 'C': 9629.023543807332, 'penalty': 'l2', 'tol': 0.0004225314724268263, 'thr
eshold': 0.08891469154343186}. Best is trial 15 with value: 0.5913552185267388.
[I 2025-12-08 23:10:48,068] Trial 16 finished with value: 0.5914334648444878 and parameters: {'fit_intercept': True, 'intercept_scaling': 3.43042135257933, 'max_iter': 1508, 'C': 9671.941753107718, 'penalty': 'l2', 'tol': 0.00036472262450446864, 'thre
shold': 0.3739777057415845}. Best is trial 16 with value: 0.5914334648444878.
[I 2025-12-08 23:12:29,877] Trial 17 finished with value: 0.5911212309627587 and parameters: {'fit_intercept': True, 'intercept_scaling': 3.318550207487302, 'max_iter': 1509, 'C': 9986.983852791756, 'penalty': 'l2', 'tol': 0.00032667384786091893, 'thr
eshold': 0.3686457188958481}. Best is trial 16 with value: 0.5914334648444878.
[I 2025-12-08 23:12:46,452] Trial 18 finished with value: 0.09787576242053221 and parameters: {'fit_intercept': False, 'intercept_scaling': 3.467912884389538, 'max_iter': 1697, 'C': 9680.776090915462, 'penalty': 'l1', 'tol': 3.694683612728508e-05, 'th
reshold': 0.6665356393077189}. Best is trial 16 with value: 0.5914334648444878.
[I 2025-12-08 23:14:23,777] Trial 19 finished with value: 0.5910060781402949 and parameters: {'fit_intercept': True, 'intercept_scaling': 2.159101919695132, 'max_iter': 1417, 'C': 8855.574462098628, 'penalty': 'l2', 'tol': 0.0004133958675816868, 'thres
hold': 0.16405553597722994}. Best is trial 16 with value: 0.5914334648444878.
Лучшие параметры для модели: {'fit_intercept': True, 'intercept_scaling': 3.43042135257933, 'max_iter': 1508, 'C': 9671.941753107718, 'penalty': 'l2', 'tol': 0.00036472262450446864, 'threshold': 0.3739777057415845}
Лучшая метрика (f1_score): 0.5914
```

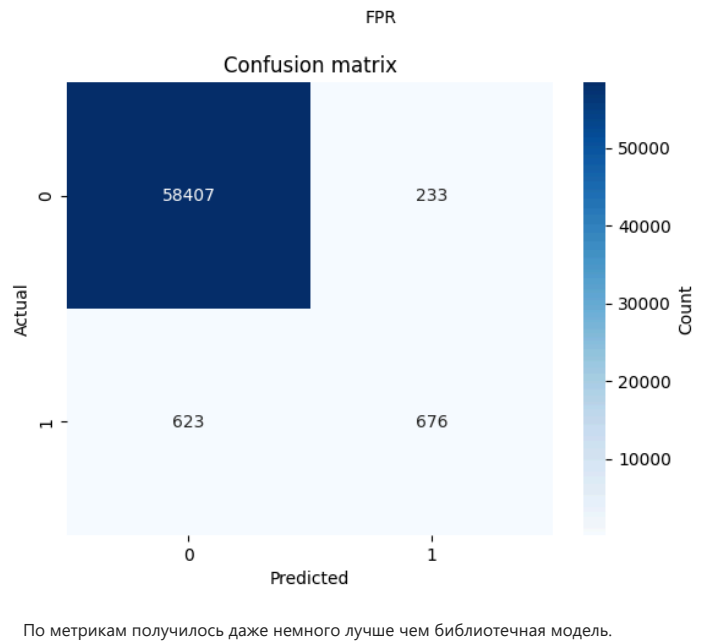
```
Out[63]: <__main__.MyLogisticRegression at 0x288c806ffa0>
```

In [64]:
```python
y_pred_class = best_model.predict(X_test)
y_pred_proba = best_model.predict_proba(X_test)
```

In [65]:
```python
evaluate(y_test=y_test, y_pred_class=y_pred_class, y_pred_proba=y_pred_proba, round_num=round_num)
```

```
ROC AUC: 0.9525

Precision: 0.7437
Recall: 0.5204
F1-Score: 0.6123
```





По метрикам получилось даже немного лучше чем библиотечная модель.

Выводы

После тюнинга с Optuna кастомные версии ElasticNet и LinerRegression показали сравнимые с оригинальными моделями результаты. В LinearRegression использовался solver liblinear как референс.